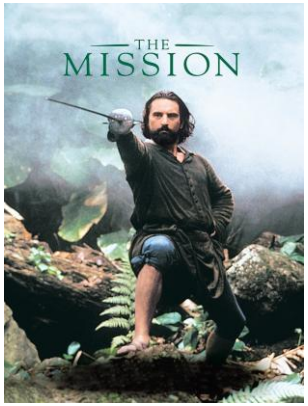
MACHINE LEARNING 2

Ciência de Dados para Negócios

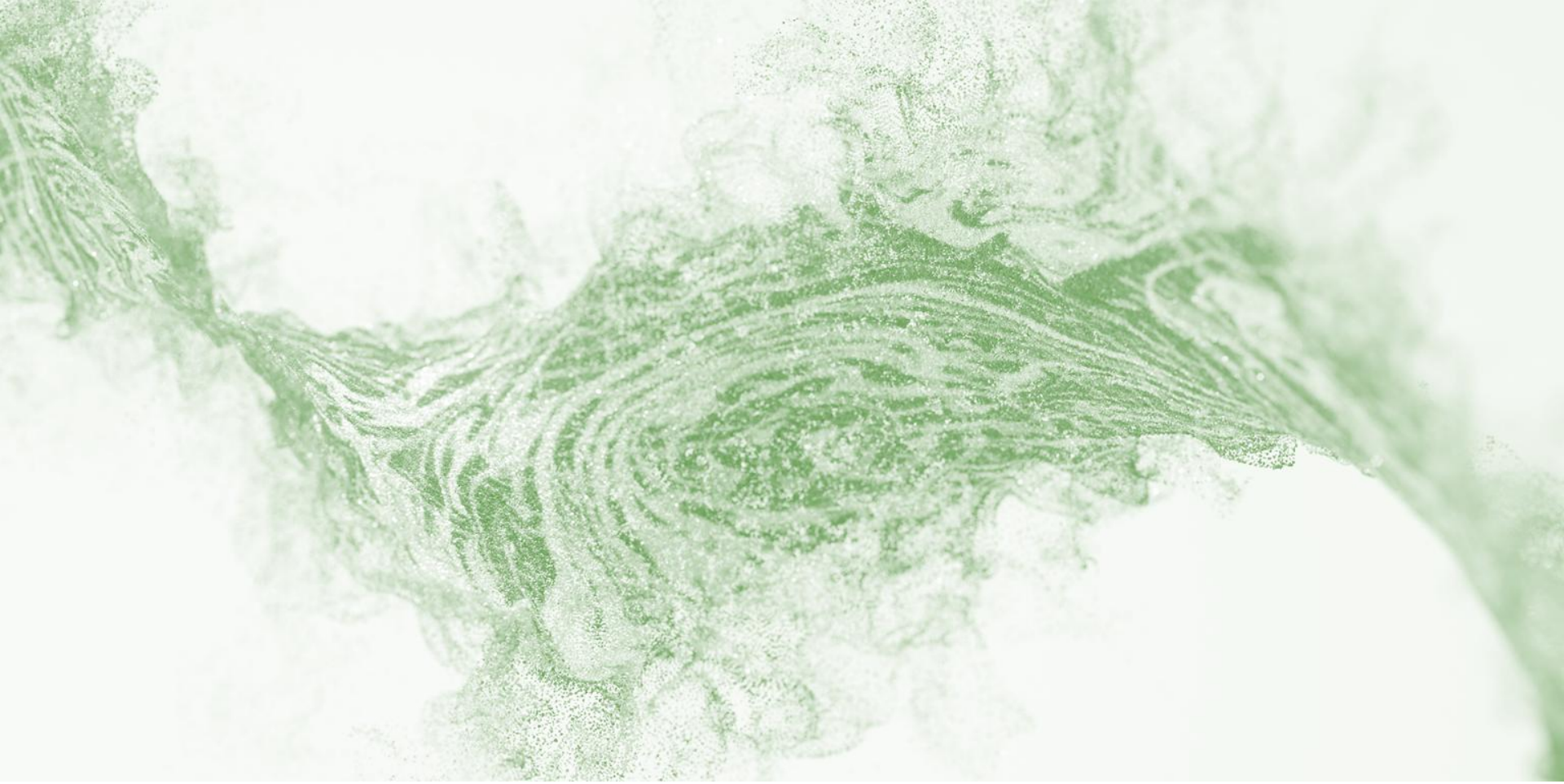
FATEC Votorantim



Prof. Luis Claudio

M. Sc. Eng. Elétrica(PUC-Rio-06)

Eng. de Computação (ITA-99)



I – INTRODUÇÃO ÀS REDES NEURAIS ARTIFICIAIS

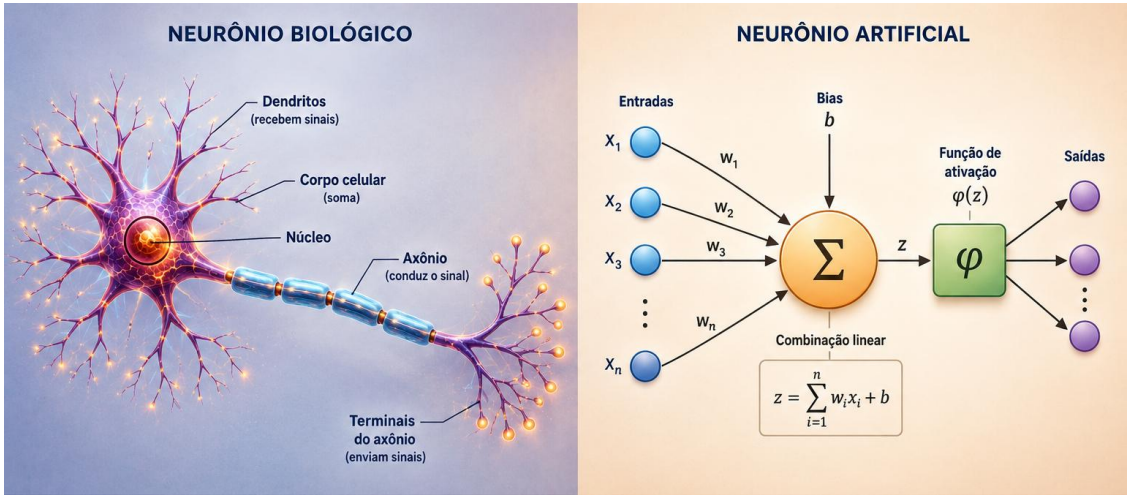
Prof. Luis Claudio
M.Sc. Eng. Elétrica(PUC-Rio-06)
Eng. de Computação (ITA-99)

O QUE É UMA REDE NEURAL

Na década de 1940, pesquisadores como **Warren McCulloch** e **Walter Pitts** tentaram responder à pergunta:

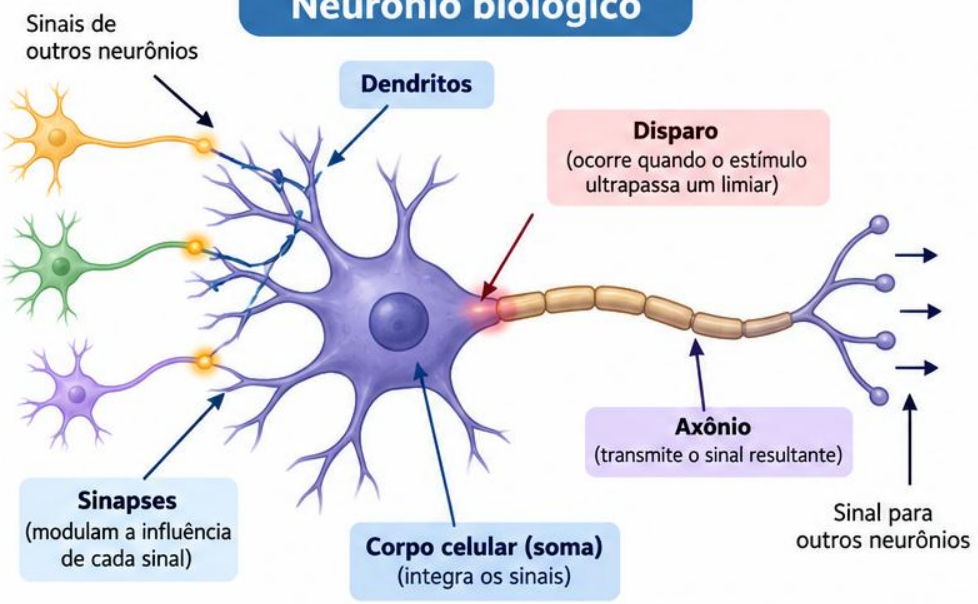
> **"Como o cérebro consegue aprender e processar informações a partir de bilhões de neurônios interconectados?"**

A ideia foi criar um modelo matemático simplificado do neurônio biológico:

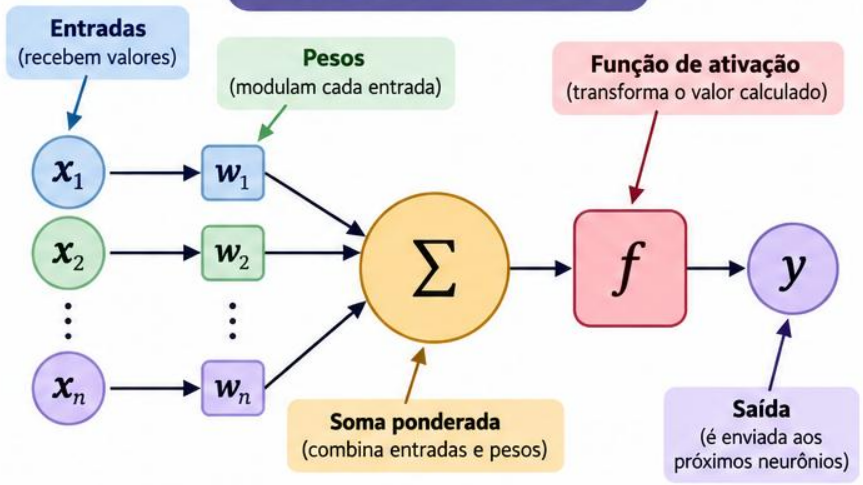


Neurônio biológico	Neurônio artificial
Dendritos recebem sinais de outros neurônios	**Entradas** $(x_1, x_2, \ldots, x_n)$ recebem valores
Sinapses modulam a influência de cada sinal	**Pesos** $(w_1, w_2, \ldots, w_n)$ modulam cada entrada
Corpo celular (soma) integra os sinais	**Soma ponderada** combina entradas e pesos
Disparo ocorre quando o estímulo ultrapassa um limiar	**Função de ativação** transforma o valor calculado
Axônio transmite o sinal resultante	**Saída** (y) é enviada aos próximos neurônios

Neurônio biológico



Neurônio artificial



$$z = \sum_{i=1}^{n} w_i x_i + b \quad \rightarrow \quad y = f(z)$$

NEURÔNIO ARTIFICIAL

O neurônio artificial é a **unidade básica de processamento** de uma rede neural. Ele recebe um conjunto de valores de entrada, realiza uma **combinação linear por meio de pesos** (*weights*) e de **um *bias***, e aplica uma **função de ativação** para produzir sua saída.

Matematicamente, o neurônio calcula a entrada líquida:

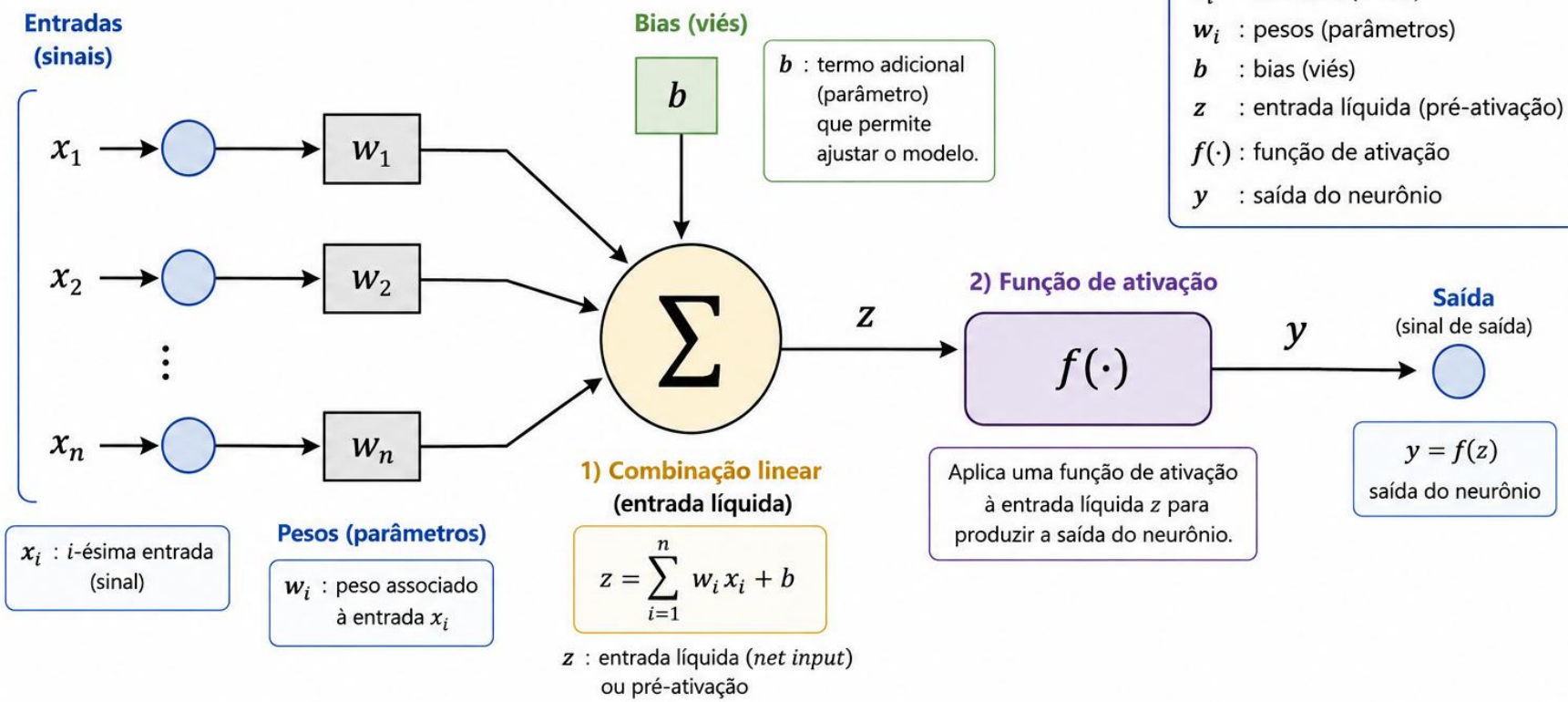
$$z = \sum_{i=1}^{n} w_i x_i + b$$

Em seguida, aplica uma função de ativação:

$$y = f(z)$$

A saída y é então encaminhada para os neurônios da camada seguinte **OU**, caso seja a última camada oculta, utilizada na camada de saída.

NEURÔNIO ARTIFICIAL



Legenda

- x_i : entradas (sinais)
- w_i : pesos (parâmetros)
- b : bias (viés)
- z : entrada líquida (pré-ativação)
- $f(\cdot)$: função de ativação
- y : saída do neurônio

b : termo adicional (parâmetro) que permite ajustar o modelo.

x_i : i-ésima entrada (sinal)

Pesos (parâmetros)

w_i : peso associado à entrada x_i

1) Combinação linear (entrada líquida)

$$z = \sum_{i=1}^{n} w_i x_i + b$$

z : entrada líquida (*net input*) ou pré-ativação

Aplica uma função de ativação à entrada líquida z para produzir a saída do neurônio.

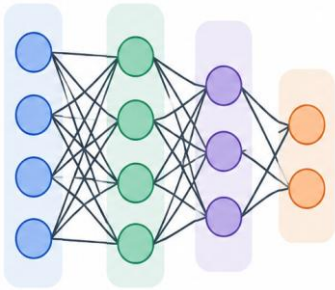
$y = f(z)$
saída do neurônio

Como funciona:

1. Cada entrada x_i é multiplicada pelo seu peso w_i.
2. Os valores ponderados são somados e o bias b é adicionado, produzindo z.
3. A função de ativação $f(\cdot)$ transforma z na saída y.
4. A saída y é enviada para os neurônios da próxima camada ou usada como saída final da rede.

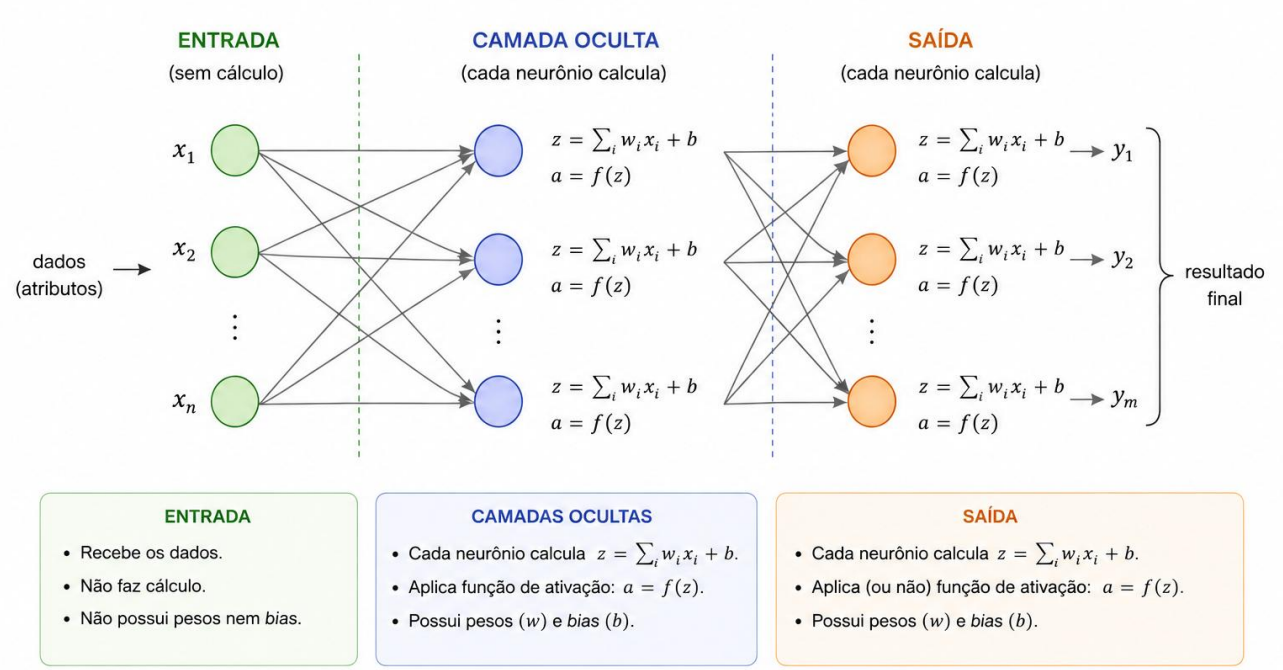
Observação histórica: Em modelos inspirados no neurônio biológico, o valor z já foi chamado de "potencial de ativação". Hoje, os termos mais comuns são entrada líquida, pré-ativação ou simplesmente z.

AS CAMADAS

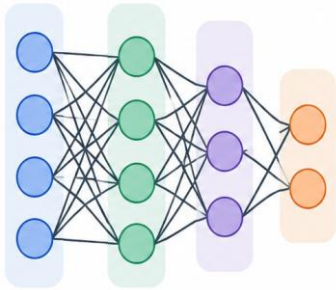


Redes neurais são arquiteturas de **aprendizagem de máquina** compostas por **neurônios artificiais organizados em camadas,** capazes de aprender representações e padrões a partir dos dados.

Em redes *feedforward* convencionais (como MLPs), distinguimos três tipos de camadas: **Entrada (Input),** uma ou mais **Ocultas (Hidden)** e **Saída (Output).**



A CAMADA DE ENTRADA



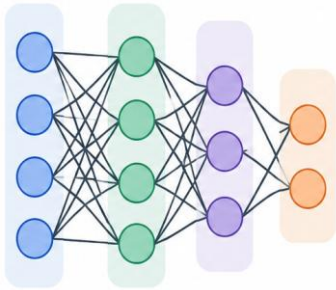
A camada de **Entrada** (*input layers*) recebe os dados de entrada (atributos), representados de **forma numérica** (dados tabulares, pixels de uma imagem etc.).

Em uma MLP tradicional (*), a camada de entrada apenas recebe os dados e os encaminha para a primeira camada oculta. Ela **não realiza processamento matemático** (**NÃO** possui pesos, bias ou função de ativação).

Por esse motivo, alguns autores **não contabilizam a camada de entrada** ao medir a **profundidade** (*depth*) da rede neural.

* Em arquiteturas mais avançadas, a camada de entrada pode ser seguida por transformações específicas, realizadas por outras camadas, como uma camada de *embedding* em problemas de PLN.

AS CAMADAS OCULTAS



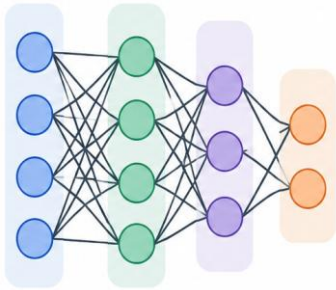
Nas camadas **Ocultas** (*hidden layers*), cada neurônio recebe as saídas da camada anterior, calcula uma combinação linear utilizando **pesos** e **bias** ($z = \sum_{i=1}^{n} w_i x_i + b$) e aplica uma **função de ativação** para produzir sua saída.

A **quantidade de camadas ocultas** e o **número de neurônios em cada camada** são **hiperparâmetros** definidos durante o projeto da rede.

As camadas ocultas quase **sempre utilizam funções de ativação**, que introduzem **não linearidade** e permitem que a rede aprenda **representações e relações complexas** a partir dos dados.

Sem funções de ativação nas camadas ocultas, sucessivas transformações lineares equivalem a uma única transformação linear, **limitando a capacidade da rede de modelar relações complexas.**

A CAMADA DE SAÍDA



Na camada de **Saída** (*output layer*) os neurônios também calculam uma combinação linear utilizando **pesos** e **bias** ($z = \sum_{i=1}^{n} w_i x_i + b$). Ou seja, ela **também tem pesos e bias aprendidos durante o treinamento**.

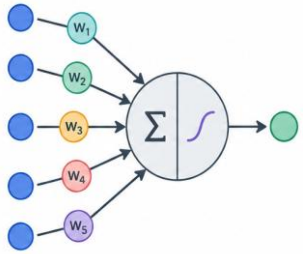
A camada de saída também usa **funções de ativação**. Mas, com o objetivo de transformar a resposta final do modelo a uma representação mais adequada.

A **função de ativação** depende do tipo de problema, como regressão (identidade), classificação binária (sigmoide) ou multiclasse (softmax).

Em arquiteturas tradicionais*, a **camada de saída é única**, embora possa conter **um ou vários neurônios** também conforme a tarefa.

* Em arquiteturas mais avançadas, a rede pode possuir **múltiplas cabeças de saída (*multiple output heads*)**, geralmente compartilhando uma estrutura comum (*backbone*).

NEURÔNIO ARTIFICIAL



Entradas (x) – Conjunto de valores fornecidos ao neurônio. Na primeira camada oculta, correspondem aos atributos de entrada; nas camadas seguintes, correspondem às saídas (ativações) da camada anterior.

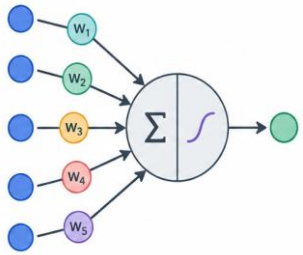
Pesos (w) – Parâmetros **aprendidos** durante o treinamento que determinam a influência de cada entrada na saída do neurônio.

Bias (b) – Termo adicional **aprendido** durante o treinamento que ajusta a combinação linear, proporcionando maior flexibilidade ao modelo.

Pré-Ativação (z) – Resultado da combinação linear entre os valores recebidos da camada anterior, seus respectivos pesos e o bias. Este é o valor submetido à função de ativação.

$$z = \sum_{i=1}^{n} w_i x_i + b = w_1 x_1 + w_2 x_2 + w_3 x_3 + \cdots + w_n x_n + b$$

NEURÔNIO ARTIFICIAL



Função de Ativação (f) – Função aplicada à **pré-ativação (z)** para produzir a **ativação (a)** do neurônio. Introduz **não linearidade** no modelo, permitindo representar relações complexas entre as variáveis. O tipo de função de ativação depende da camada e do problema que se deseja resolver.

Ativação (a) – Resultado produzido pelo neurônio após a aplicação da função de ativação sobre **z**. Nas camadas ocultas, constitui a **saída do neurônio**, sendo enviada como entrada aos neurônios da camada seguinte.

$$\text{neurônio oculto: } z \rightarrow f(z) \rightarrow a$$

Saída do modelo ($\hat{y}$) – Resultado produzido pela **camada de saída** da rede. Representa a previsão final do modelo e sua interpretação depende do problema, podendo corresponder, por exemplo, a um valor numérico, uma probabilidade ou uma distribuição de probabilidades.

$$\text{camada de saída: } z \rightarrow f(z) \rightarrow \hat{y}$$

Neurônio Artificial aplicado a dados tabulares

1 Dados tabulares

	idade	renda	score	...
1	25	3000	650	...
2	40	8000	700	...
3	35	5000	620	...
⋮	⋮	⋮	⋮	⋮
n	28	4500	680	...

- Cada linha = uma instância (exemplo)
- Cada coluna (de entrada) = uma característica

2 Após transformações / padronização

x_1	x_2	x_3	...	x_p
−0,45	−1,10	0,20	...	0,35
0,85	1,05	1,10	...	−0,15
0,30	−0,10	−0,30	...	0,60
⋮	⋮	⋮	⋮	⋮
−0,20	−0,60	0,40	...	0,10

Cada linha vira um vetor de **entrada**

$$x = [x_1, x_2, \ldots, x_p]$$

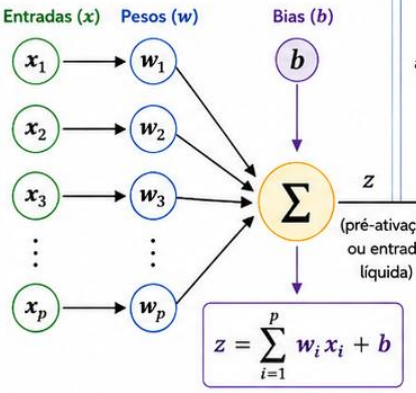
3 Entradas do neurônio (uma instância)

A linha selecionada fornece os valores numéricos que alimentam o neurônio.

$x_1 = 0{,}30$
$x_2 = -0{,}10$
$x_3 = -0{,}30$
⋮
$x_p = 0{,}60$

Entradas (x): valores numéricos da instância selecionada

4 Componentes do neurônio artificial



Pesos (w): determinam a influência de cada entrada

Bias (b): termo de ajuste adicional

Pré-ativação (z): combinação linear das entradas com pesos e *bias*

5 Função de ativação

Transforma z na ativação do neurônio.

$f(z)$

Função de ativação f: introduz não linearidade ao modelo.

6 Saída / ativação do neurônio

$a = f(z)$

a é o valor produzido pelo neurônio.

Em camadas ocultas, a é enviada à próxima camada; na camada de saída, contribui para a resposta final da rede.

Resumo do fluxo de uma instância

Linha da tabela (instância selecionada) → **Vetor de entrada** $x = [x_1, x_2, \ldots, x_p]$ → **Multiplicação** $x_i \cdot w_i$ → **Soma + bias** $\sum_{i=1}^{p} w_i x_i + b$ → **Pré-ativação** z (entrada líquida) → **Ativação** $a = f(z)$

Linha da tabela	Vetor de entrada x	Multiplicação	Soma + bias	Pré-ativação z	Ativação
Uma linha da tabela de dados	Valores padronizados da instância	Cada entrada é multiplicada pelo seu peso	Soma dos produtos mais o *bias*	Resultado da combinação linear das entradas com pesos e *bias*	Aplicação da função de ativação

Onde cada elemento atua:

- **Entradas (x)**: representam os valores da instância
- **Pesos (w)**: controlam a contribuição de cada entrada
- **Bias (b)**: desloca a combinação linear
- **Função de ativação f**: introduz não linearidade
- **Ativação (a)**: saída produzida pelo neurônio

BIAS

O **bias (b)** permite **deslocar a fronteira de decisão** em relação à origem.

Em duas dimensões, essa fronteira pode ser representada por uma **reta,** cuja forma é equivalente à expressão do neurônio $\mathbf{w_1x_1 + w_2x_2 + b = 0}$.

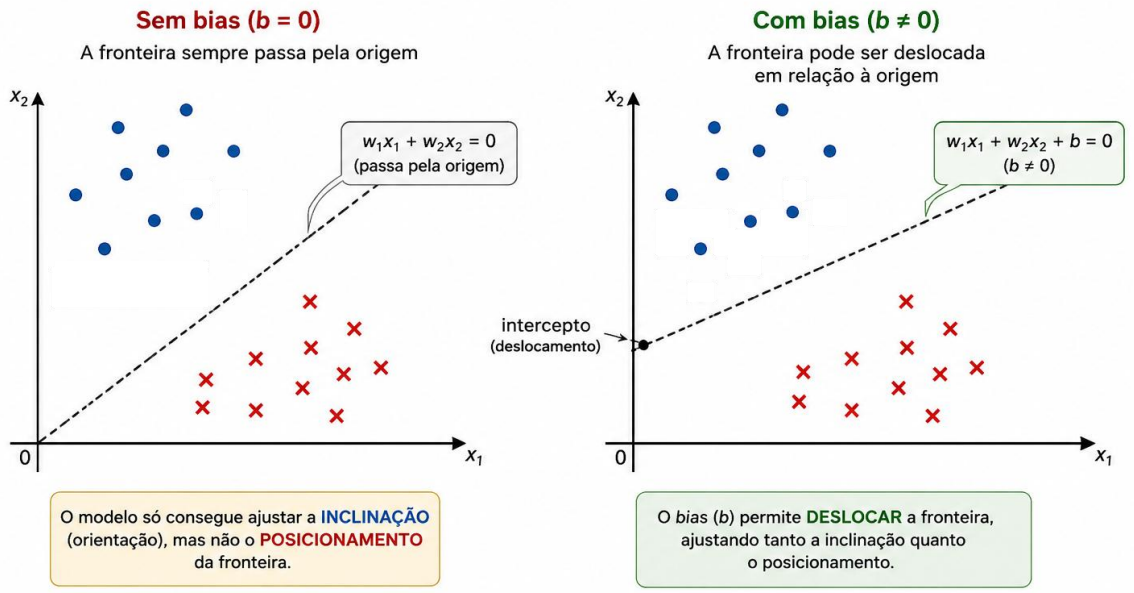
Sem o bias (b = 0), a fronteira $\mathbf{w_1x_1 + w_2x_2 = 0}$ sempre passa pela origem.

Com bias (b ≠ 0), a fronteira pode ser deslocada em relação à origem.

$$w_1x_1 + w_2x_2 + b = 0$$

$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{b}{w_2}$$

$$\underbrace{x_2}_{y} = \underbrace{\left(-\frac{w_1}{w_2}\right)}_{a}\underbrace{x_1}_{x} + \underbrace{\left(-\frac{b}{w_2}\right)}_{\text{intercepto}}$$



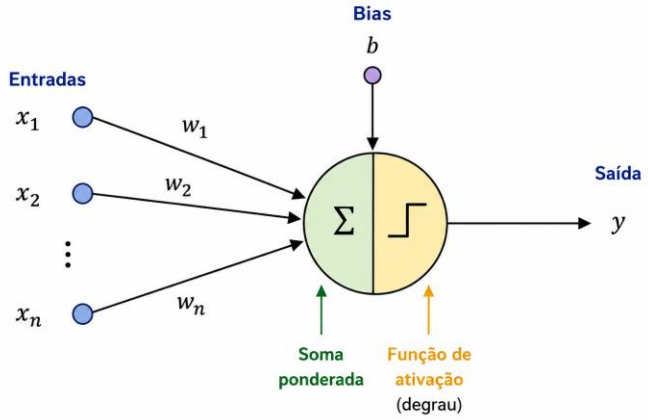


PERCEPTRON
MODELO DE ROSENBLATTMO

Perceptron foi proposto por **Frank Rosenblatt em 1958** e é considerado um dos primeiros modelos de neurônio artificial com **capacidade de aprendizado**. Em sua forma mais simples, pode ser representado como uma **rede neural baseada em um único neurônio**.

O Perceptron tornou-se um dos modelos mais importantes na história das Redes Neurias por introduzir o **aprendizado de máquinas.**

O **Perceptron** simples foi utilizado em problemas de **classificação binária** (**0 e 1, sim e não** ou **verdadeiro e falso**). Ele utilizava uma **função de ativação degrau** (*step* ou *threshold*) e era capaz de resolver problemas **linearmente separáveis.**





PERCEPTRON
LIMITAÇÃO

O **Perceptron realiza classificação linear,** aprendendo uma **fronteira de decisão** que separa duas classes por uma **reta (2D), plano (3D) ou hiperplano (múltiplas dimensões).** Essa fronteira é definida pela **combinação linear dos atributos de entrada.**

$$z = \sum_i w_i x_i + b$$

Embora a **função degrau seja não linear,** ela apenas identifica de qual lado da fronteira está a entrada. A limitação está na capacidade de um **único neurônio,** que produz apenas uma **fronteira linear** e, portanto, resolve somente problemas **linearmente separáveis.**

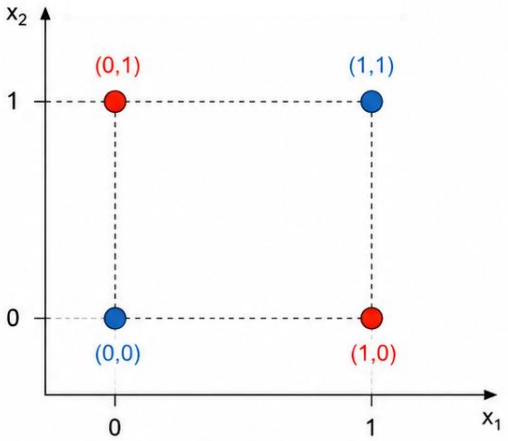
O **XOR** tornou-se o exemplo clássico dessa limitação: **não existe uma única reta capaz de separar corretamente suas duas classes.**



PERCEPTRON
LIMITAÇÃO

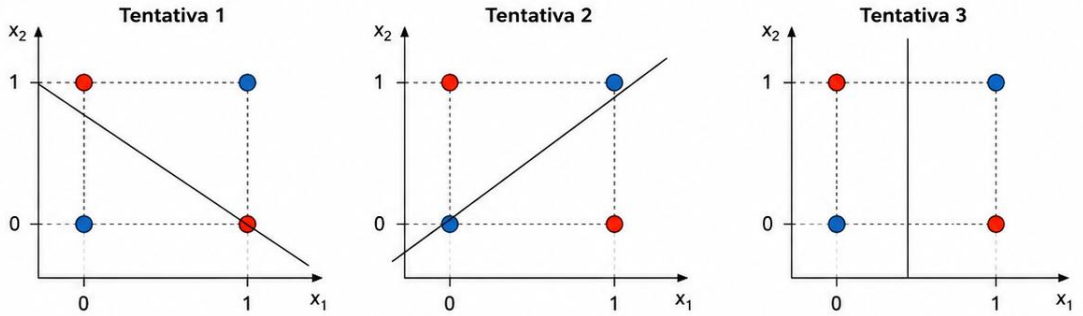
Problema XOR

x_1	x_2	x_1 XOR x_2 (saída)
0	0	0
0	1	1
1	0	1
1	1	0

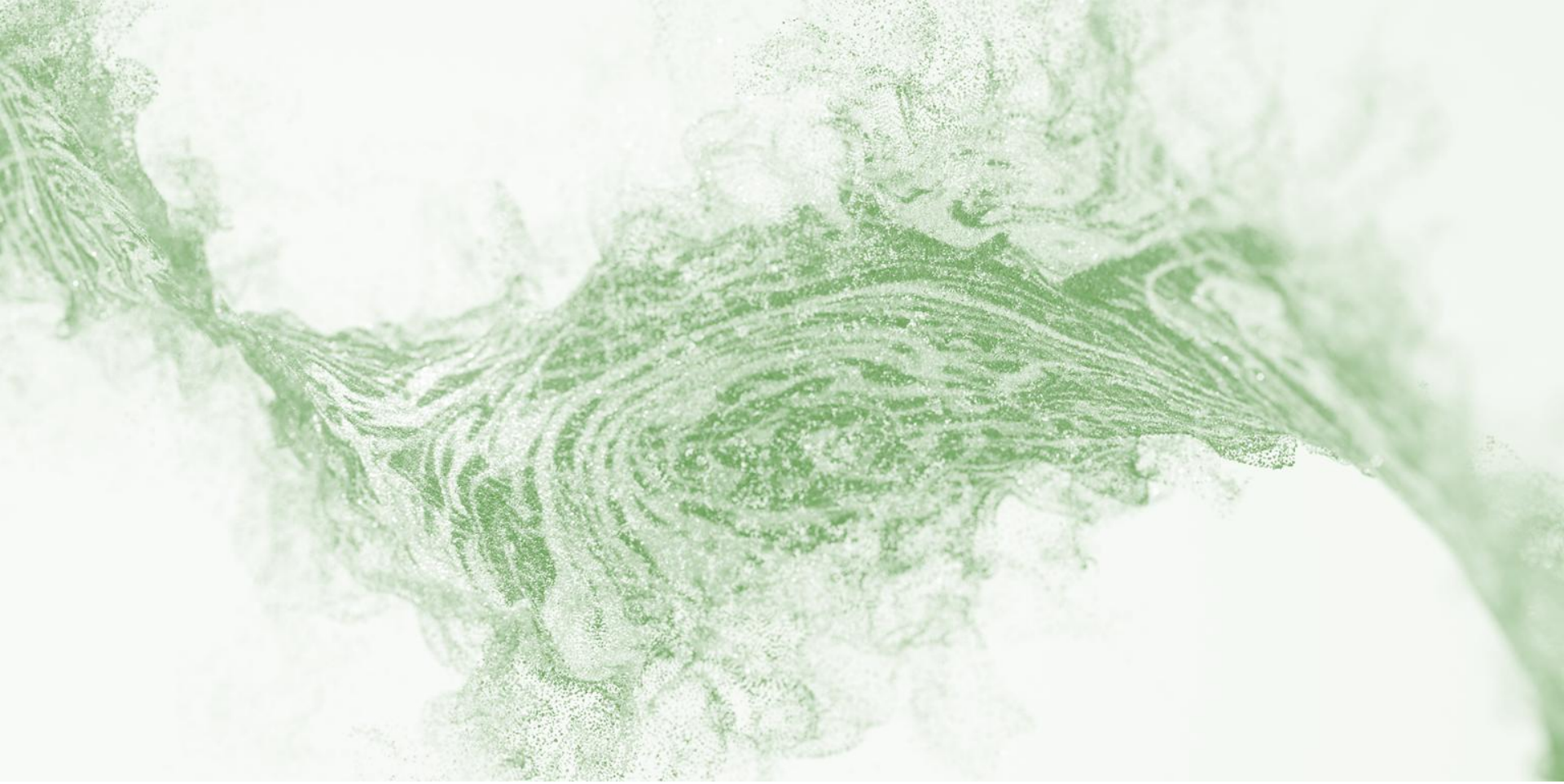


Classe 0 (saída = 0)
Classe 1 (saída = 1)

Por que o perceptron (classificação linear) não resolve?



✗ Nenhuma reta (ou hiperplano) consegue separar as classes corretamente.



II – TREINAMENTO DE REDES NEURAIS

Prof. Luis Claudio
M.Sc. Eng. Elétrica(PUC-Rio-06)
Eng. de Computação (ITA-99)

HIPERPARÂMETROS

Parâmetros

São **aprendidos** pela rede durante o treinamento. Representam **o que** o modelo aprende a partir dos dados.

- Exemplos: **pesos (weights)** e **vieses (biases)**.

Hiperparâmetros

São **definidos pelo cientista de dados**.

Determinam **como** e em **quais condições** o aprendizado ocorrerá.

- Exemplos: número de camadas (“profundidade”), número de neurônios por camada (“largura”), função de ativação, tamanho de lote (*batch size*), número de épocas, função de perda (*loss function*), otimizador e taxa de aprendizado (*learning rate*) entre outras.

HIPERPARÂMETROS
ARQUITETURA VERSUS TREINAMENTO

Embora não seja uma divisão canônica, eu gosto de dividir em:

Hiperparâmetros de Arquitetura

São paramentos que definem **como a rede neural é estruturada**:

- Número de camadas ocultas
- Número de neurônios por camada oculta
- Função de ativação

Hiperparâmetros de Treinamento

São parâmetros que definem **como o aprendizado acontecerá**:

- Tamanho do lote (*batch size*)
- Número de épocas (*epochs*)
- Função de perda (*loss function*)
- Otimizador
- Taxa de aprendizado (*learning rate*)


ReLU / tanh /
sigmoid
HIPERPARÂMETROS
DE ARQUITETURA
(estrutura da rede)



HIPERPARÂMETROS
QTD DE CAMADAS OCULTAS

Não existe uma regra que determine o número ideal de camadas ocultas.

A escolha inicial depende principalmente da **complexidade do problema** e deve ser **avaliada experimentalmente**.

Em geral, quanto **mais camadas, maior a capacidade** da rede de construir representações mais complexas, mas também **maior o custo computacional**.

Na pratica...

- Para problemas tabulares tradicionais, **1 a 3 camadas ocultas** costumam ser um ponto de partida razoável.
- Redes com dezenas ou centenas de camadas estão normalmente associadas a arquiteturas e problemas muito mais complexos (LLMs, por exemplo).



HIPERPARÂMETROS
QTE DE NEURÔNIOS POR CAMADA

Diferentemente das camadas de entrada e saída, cuja quantidade de neurônios está diretamente relacionada, respectivamente, à dimensionalidade dos dados e ao tipo de problema, também **não existe uma regra que determine a quantidade ideal de neurônios nas camadas ocultas.**

Aqui também, a escolha inicial é uma **hipótese de arquitetura**, posteriormente validada experimentalmente.

Na prática...

- É comum utilizar valores como **32, 64, 128, 256**, por convenção e também por eficiência computacional (legado).
- A arquitetura pode manter, aumentar ou reduzir a quantidade de neurônios entre as camadas. Um **afunilamento progressivo** é uma estratégia bastante utilizada (mas existem outras, como “V”, “W” etc.).
- Mais neurônios aumentam a **capacidade de representação** da rede, mas também aumentam o **número de parâmetros e o custo computacional**.



HIPERPARÂMETROS
FUNÇÕES DE ATIVAÇÃO

As funções de ativação são aplicadas ao valor de pré-ativação (**z**).

Nas **camadas ocultas**, sua principal função é **introduzir não linearidade**, permitindo que a rede aprenda relações e fronteiras de decisão complexas, que não poderiam ser representadas por sucessivas transformações lineares.

Entre as principais funções de ativação estão:

- Sigmoid
- Tanh (Tangente Hiperbólica)
- ReLU

Há diversas outras, como **Leaky ReLU, ELU, GELU, SiLU/Swish e Softplus.**



HIPERPARÂMETROS
FUNÇÕES DE ATIVAÇÃO

By the way, uma função linear é uma combinação do tipo:

$$f(x_1, \ldots, x_n) = a_1 x_1 + a_2 x_2 + \cdots + a_n x_n$$

Logo:

- cada variável aparece em expoente 1;
- não há produto entre variáveis, como $x_1 x_2$;
- não há potências como x^2, x^3;
- não há $\sqrt{x}$, $1/x$, e^x, $\log(x)$, $\sin(x)$, etc.;
- preserva a linearidade: $f(x_1 + x_2) = f(x_1) + f(x_2)$ e $f(cx) = c\,f(x)$

Exemplos:

$2x_1 - 3x_2$ **é linear**

$2x_1 x_2$; $x_1^2 + x_2$; $\sin(x_1) + x_2$ **não são lineares**



FUNÇÃO DE ATIVAÇÃO
SIGMOIDE

A função sigmoide recebe qualquer valor de **z entre -∞ e +∞** e produz uma saída entre **0 e 1**. É uma função **não linear,** contínua e com formato de "S" que produz uma **limitação/compressão** da faixa das ativações.

É **centrada em 0,5** (para **z = 0,** a saída é **0,5**); para **z ≈ 2,2,** a saída é aproximadamente **0,9**; para **z ≈ -2,2,** aproximadamente **0,1.**

$$f(z) = \frac{1}{1 + e^{-z}}$$

Para valores maiores de |z|, a função entra em uma região de **saturação,** na qual grandes alterações em z produzem pequenas alterações na saída. Essa saturação pode provocar o problema de ***vanishing gradiente*** (gradiente que desaparece), dificultando o treinamento de redes mais profundas*.

* Na retropropagação, para calcular o **gradiente da função de perda em relação aos pesos,** usamos a **regra da cadeia,** que envolve as **derivadas das funções de ativação**. Por isso a saturação é problemática: uma curva quase horizontal gera derivada ≈ 0 e o gradiente tende a zero, prejudicando a atualização dos pesos (em outras palavras "a rede para de aprender").

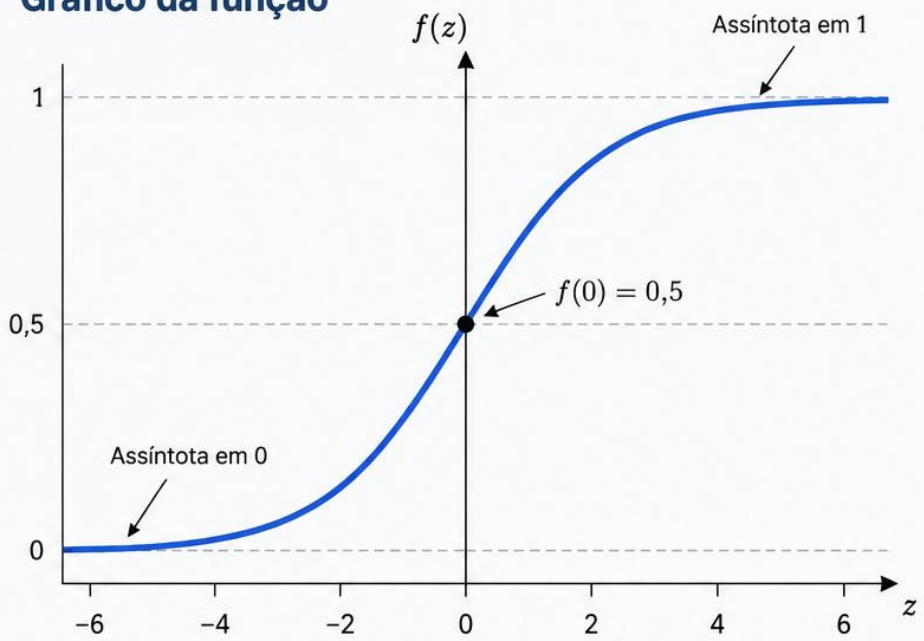
Função Sigmoide

$$f(z) = \frac{1}{1 + e^{-z}}$$

Principais características

- Recebe qualquer valor de z (de $-\infty$ a $+\infty$) e produz uma saída entre 0 e 1.
- É uma função não linear, contínua e com formato de "S".
- Para $z = 0$, a saída é 0,5.
- Para valores elevados de $|z|$, a função entra em uma região de saturação.
- A saturação pode provocar o problema de vanishing gradient, dificultando o treinamento de redes mais profundas.
- Atualmente, é pouco utilizada em camadas ocultas, sendo amplamente substituída pela ReLU e suas variantes.

Gráfico da função



Exemplos de valores

z	−4	−2	0	2	4
$f(z)$	0,018	0,119	0,500	0,881	0,982



FUNÇÃO DE ATIVAÇÃO
TANGENTE HIPERBÓLICA

A função tanh recebe qualquer valor de **z entre -∞ e +∞** e produz uma saída entre **-1 e 1**. Também é **não linear**, contínua e com formato de "S" e que também produz uma **limitação/compressão** da faixa das ativações.

É **centrada em 0** (para **z = 0**, a saída é **0**); para **z ≈ 1,47**, a saída é aproximadamente **0,9**; para **z ≈ -1,47**, aproximadamente **-0,9**.

$$f(z) = \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

Para valores maiores de |z|, também apresenta **saturação**, aproximando-se de -1 ou 1. Ou seja, também está sujeita ao ***vanishing gradient***.

Uma vantagem da tanh em relação à sigmoid é produzir saídas centradas em zero, **preservando valores negativos e positivos**.

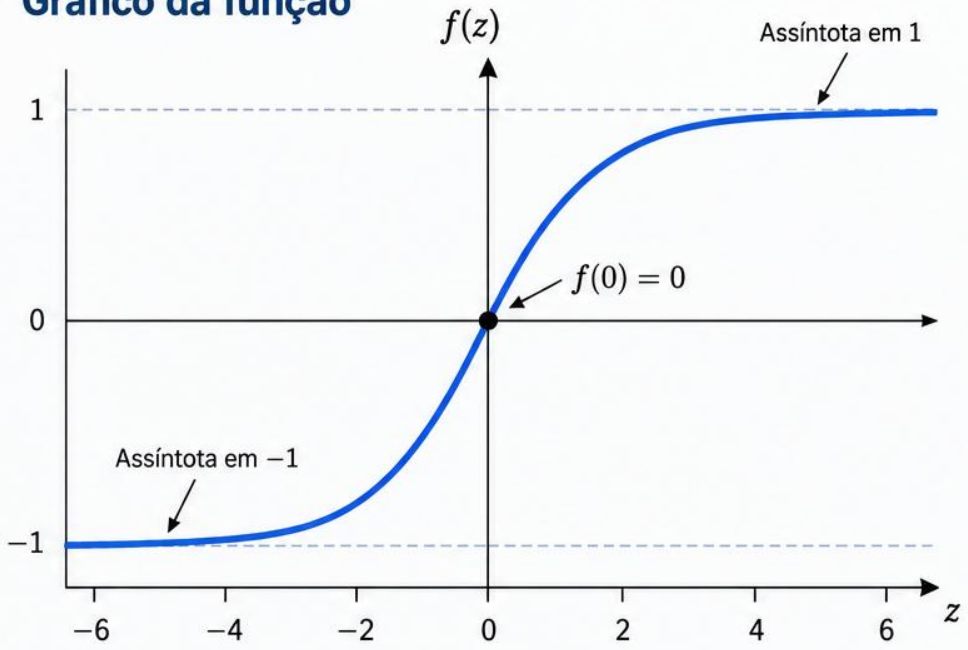
Função Tangente Hiperbólica (*Tanh*)

$$f(z) = \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

Principais características

- Recebe qualquer valor de z (de $-\infty$ a $+\infty$) e produz uma saída entre -1 e 1.
- É uma função não linear, contínua e com formato de "S".
- É simétrica em relação à origem: $tanh(-z) = -tanh(z)$.
- Para $z = 0$, a saída é 0.
- Para valores elevados de $|z|$, a função entra em uma região de saturação (próxima de -1 ou 1).
- Também pode sofrer com o problema de vanishing gradient, embora em menor grau que a função sigmoide.
- Foi amplamente utilizada em redes neurais, sendo hoje menos comum devido ao melhor desempenho da ReLU e suas variantes.

Gráfico da função



Exemplos de valores

z	−4	−2	0	2	4
$tanh(z)$	−0,999	−0,964	0,000	0,964	0,999



HIPERPARÂMETROS
FUNÇÃO RELU

ReLU (*Rectified Linear Unit*) é atualmente uma das funções de ativação mais utilizadas em **camadas ocultas**.

Seu comportamento é **extremamente simples**:

- Para **z ≤ 0**, a saída é **0**;
- Para **z > 0**, a saída é o **próprio z**.

$$f(z) = \max(0, z)$$

Apesar de sua simplicidade, é **não linear**, pois apresenta uma mudança de comportamento em **z = 0**.

- Em **z=0**, a derivada **não é matematicamente definida**; nas implementações computacionais, adota-se convencionalmente um valor para permitir o treinamento.
- Para **z > 0**, sua derivada é **1**, evitando a saturação observada na sigmoide. Isso contribui para reduzir o problema de ***vanishing gradient***.

* Como limitações, a ReLU **não limita superiormente as ativações**, que podem assumir valores muito elevados e contribuir para instabilidades no treinamento. Além disso, para z<0, sua derivada é 0, podendo ocorrer o problema conhecido como **dying ReLU**. Variantes como **Leaky ReLU** introduzem uma pequena inclinação para valores negativos, buscando reduzir este último problema.

ReLU — Rectified Linear Unit

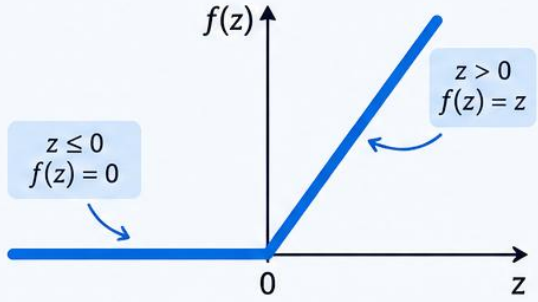
(Unidade Linear Retificada)

- Uma das funções de ativação mais utilizadas em **camadas ocultas.**
- Simples e eficiente.
- É **não linear**, pois muda o comportamento em z = 0.
- Para z > 0, sua derivada é 1, evitando a saturação e ajudando a reduzir o problema de **vanishing gradient.**

Definição

$$f(z) = \max(0, z)$$

Gráfico



Exemplos de valores

z	−4	−2	0	2	4
$f(z)$	0	0	0	2	4
$f'(z)$	0	0	(indef.)	1	1

Obs.: em z = 0, a derivada não é definida. Em implementações práticas, costuma-se usar 0 ou 1.

Vantagens

- Não satura para z > 0 (derivada = 1)
- Ajuda a reduzir o vanishing gradient
- Computacionalmente simples e eficiente
- Funciona bem em redes profundas

Limitações

- Derivada = 0 para z < 0
- Pode causar o problema de "dying ReLU" (neurônios que param de ativar)

Em resumo: a ReLU é simples, eficiente e evita a saturação na região positiva, o que facilita o treinamento de redes profundas. Por isso, é hoje a função de ativação mais usada em camadas ocultas.



HIPERPARÂMETROS
FUNÇÃO RELU

A **ReLU tornou-se uma função padrão para camadas ocultas,** especialmente em redes profundas, porque é **simples, computacionalmente barata e reduz o problema** de *vanishing gradient* na região positiva.

$$f(z) = \max(0, z)$$

Mas hoje o cenário é mais diversificado:

- **ReLU e variantes** → muito comuns em CNNs e diversas redes *feedforward*.
- **GELU** → tornou-se muito comum em **Transformers.**
- **SiLU/Swish** → utilizada em arquiteturas modernas, inclusive redes de visão.



HIPERPARÂMETROS
FUNÇÕES DE ATIVAÇÃO NA SAÍDA

Na **camada de saída**, sua principal função é **adequar a saída da rede ao tipo de problema que está sendo modelado e ao formato esperado da resposta**.

Dessa forma, a função de ativação pode determinar características da saída, como seu **intervalo de valores e sua interpretação**.

A escolha da função de ativação na camada de saída está, portanto, diretamente relacionada ao **tipo de problema**, devendo também ser **compatível com a função de perda** utilizada durante o treinamento.



HIPERPARÂMETROS
FUNÇÕES DE ATIVAÇÃO NA SAÍDA

A escolha da função de ativação e da quantidade de neurônios da **camada de saída** depende do tipo de problema:

- **Regressão:** geralmente **1 neurônio** com função **Identidade (Linear)**. A saída pode assumir valores contínuos sem limitação de intervalo.
- **Classificação Binária:** geralmente **1 neurônio** com **Sigmoid**. A saída varia entre **0 e 1** e pode ser interpretada como a **probabilidade da classe positiva**.
- **Classificação Multiclasse:** geralmente **1 neurônio por classe** com **Softmax**. As saídas representam probabilidades associadas às classes e **somam 1**.
- **Classificação Multilabel:** geralmente **1 neurônio por classe**, com **Sigmoid aplicada independentemente a cada neurônio**. Cada classe é tratada como um problema de **"Sim ou Não"**, permitindo **múltiplas classes simultaneamente**. **Exemplo:** uma imagem pode conter **pessoa, cachorro e carro** ao mesmo tempo.

MULTICLASSE (Softmax)

Apenas uma classe é escolhida.
As probabilidades somam 1.

Exemplo:
Qual **animal** aparece na imagem?



Classe	Softmax (probabilidade)
Cachorro	**0,91**
Gato	0,06
Cavalo	0,03

Soma = 1,00

➜ **Resultado: Cachorro**

Características:
Softmax impõe competição entre as classes: apenas **uma** será a escollhida.

MULTILABEL (Sigmoid)

Várias classes podem ser atribuídas simultaneamente.
Cada saída é independente (não somam 1).

Exemplo:
Quais **objetos** estão presentes na imagem?



Classe	Sigmoid (probabilidade)
Pessoa	**0,96**
Cachorro	**0,91**
Carro	**0,83**
Árvore	**0,12**

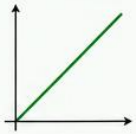
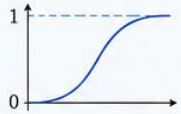
Não somam 1
(saídas independentes)

➜ **Resultado: Pessoa + Cachorro + Carro**

Características:
Cada classe é tratada como um problema de **"Sim ou Não"**.
Várias classes podem ter probabilidade alta ao mesmo tempo.

Resumo: **Multiclasse** = escolhas mutuamente exclusivas | **Multilabel** = rótulos independentes que podem coexistir.

Configuração da Camada de Saída em Redes Neurais

Tipo de Problema	Neurônios na Saída	Função de Ativação	Saída Produzida	Interpretação	Exemplo
Regressão Prever um ou mais valores contínuos (reais).	**1 (geralmente) ou mais** $\bigcirc \longrightarrow y_1$ $\bigcirc \longrightarrow y_2$ $\vdots \quad \vdots$ $\bigcirc \longrightarrow y_k$ (1 neurônio para cada variável-alvo)	**Linear (identidade)**  $f(z) = z$	**Valores reais (sem restrição)** Ex.: $y \in (-\infty, +\infty)$	Cada neurônio produz um valor real que pode assumir qualquer valor (no intervalo real).	**Exemplos:** • Preço de um imóvel • Temperatura • LTV • Prever temperatura e umidade (2 saídas) 
Classificação Binária Prever se pertence à classe positiva (1) ou negativa (0).	**1** $\bigcirc \longrightarrow y$	**Sigmoid**  $f(z) = \frac{1}{1+e^{-z}}$	**Uma probabilidade entre 0 e 1** Ex.: $y \in [0, 1]$	Representa a probabilidade da classe positiva (1). Usa-se um limiar (ex.: 0,5) para decidir a classe.	**Exemplos:** • Spam / Não spam • Fraude / Não fraude • Cliente vai comprar / Não vai comprar 
Classificação Multiclasse Existem várias classes possíveis, mas cada exemplo pertence apenas a uma delas.	**1 por classe (k classes)** $\bigcirc \longrightarrow z_1$ $\bigcirc \longrightarrow z_2$ $\vdots \quad .$ $\bigcirc \longrightarrow z_k$	**Softmax** $f(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$ (aplicada ao vetor z)	**Distribuição de probabilidades (soma = 1)** Ex.: $[0, 0.15, 0.70, 0.15]$ $(\sum_i p_i = 1)$	A classe prevista é aquela com a maior probabilidade. As probabilidades de todas as classes somam 1.	**Exemplos:** • Tipo de animal: gato, cachorro ou cavalo • Meio de transporte: carro, ônibus ou avião 
Classificação Multirrótulo (Multi-label) Um exemplo pode pertencer a várias classes ao mesmo tempo.	**1 por classe (k classes)** $\bigcirc \longrightarrow y_1$ $\bigcirc \longrightarrow y_2$ $\vdots \quad .$ $\bigcirc \longrightarrow y_k$	**Sigmoid** (em cada neurônio)  $f(z) = \frac{1}{1+e^{-z}}$	**Probabilidade independente para cada classe** Ex.: $[0.95, 0.82, 0.10, 0.67]$ (não somam 1)	Cada neurônio indica a probabilidade daquela classe estar presente, independentemente das demais.	**Exemplos:** • Em uma imagem pode haver: pessoa, carro, árvore e bicicleta simultaneamente • Doenças que um paciente pode ter 

Regra prática: A configuração da camada de saída depende do tipo de problema. A quantidade de neurônios e a função de ativação são escolhidas de acordo com o formato da resposta que se deseja produzir.


Learning rate
Batch size
Épocas
Otimizador
Regularização
HIPERPARÂMETROS
DE TREINAMENTO
(aprendizado / otimização)



HIPERPARÂMETROS
TAMANHO DO LOTE (BATCH SIZE)

O **tamanho do lote (*batch size*)** determina **quantas instâncias do conjunto de treinamento são processadas antes de cada atualização dos pesos e biases**.

Por exemplo, para um dataset com **10.000 instâncias,** escolher ***batch_size*** = 100 gera aproximadamente **100 atualizações** dos parâmetros a cada **rodada**.

É um **hiperparâmetro de treinamento** definido experimentalmente, pois **não existe uma regra universal** para determinar seu valor. Valores como **16, 32, 64, 128 e 256** são frequentes, mas *batches* de **512, 1024 ou maiores** também podem ser utilizados, especialmente em treinamentos de grande escala.

- **Batch grande:** observa muitas amostras para decidir → estima uma **direção mais estável** → porém precisa processar **muitas amostras antes de reavaliar a direção e atualizar os pesos novamente.**
- **Batch pequeno:** observa poucas amostras para decidir → estima uma **direção menos estável** → porém rapidamente pode **reavaliar a direção e fazer uma nova atualização.**



HIPERPARÂMETROS
NÚMERO DE ÉPOCAS (EPOCHS)

Uma **época** corresponde a **uma passagem completa de todo o conjunto de treinamento pela rede neural**. Quanto mais épocas, mas vezes o conjunto completo de treinamento é apresentado à rede.

Durante uma época, os dados são processados em lotes, e os parâmetros são atualizados à medida que esses lotes são processados.

Por exemplo, com **10.000 instâncias** e ***batch_size* de 100**, teremos **100 lotes por época**. Se usar como critério de parada a quantidade de épocas igual a **50**, serão realizadas **5.000 atualizações dos parâmetros**.

Obs.: Note que **poucas épocas** podem resultar em ***underfitting*** (aprendizado insuficiente) e, por outro lado, **muitas épocas** podem resultar em ***overfitting*** (além de aumentar o tempo de treinamento e custo de processamento).



HIPERPARÂMETROS
FUNÇÃO DE PERDA (LOSS FUNCTION)

A **função de perda** (*L*) é usada para **medir o erro do modelo**, comparando a saída prevista **y^** com o valor esperado **y**. Durante o treinamento, o objetivo é **minimizar o erro** por meio do **ajuste dos pesos e biases** da rede.

A forma de calcular a perda depende do **tipo de problema**. Por exemplo:

- **Regressão:**
  - **MSE** (***Mean Squared Error***) — calcula a média dos **erros ao quadrado**, penalizando mais fortemente erros maiores. Ex.: esperado = **10**, previsto = **7** → erro = $10 - 7 = 3$ → erro ao quadrado = $\mathbf{3^2 = 9}$.
  - **MAE** (***Mean Absolute Error***) — calcula a média dos **erros absolutos**, sendo menos sensível a valores extremos. Ex.: esperado = **10**, previsto = **7** → erro absoluto = $|10 - 7| = 3$.
- **Classificação Binária (Binary Cross-Entropy)** — penaliza o modelo de acordo com a **probabilidade atribuída à classe correta**, considerando duas classes; previsões confiantes e incorretas recebem perdas maiores.
- **Classificação Multiclasse (Categorical Cross-Entropy)** — penaliza o modelo de acordo com a **probabilidade atribuída à classe correta**, considerando múltiplas classes; previsões confiantes e incorretas recebem perdas maiores.



HIPERPARÂMETROS
TAXA DE APRENDIZADO

A **taxa de aprendizado** (*learning rate*) determina a **magnitude das alterações realizadas nos pesos e biases** a cada atualização durante o treinamento. Normalmente é representada por **η** (*eta*) ou **α**.

A partir da **função de perda,** são calculados os **gradientes**, que indicam a direção em que cada parâmetro deve ser alterado para reduzir o erro. A taxa de aprendizado controla o **tamanho do passo** realizado nessa direção.

Valores típicos de *learning rate* são **0.01, 0.001 e 0.0001**.

Obs.: taxas muito altas produzem **passos grandes,** podendo ultrapassar regiões de mínimo e dificultar a convergência. Taxas muito baixas produzem **passos pequenos,** tornando o aprendizado mais lento.



HIPERPARÂMETROS

*GRADIENTE (*NÃO É UM HIPERPARÂMETRO EM SI)

A função de perda pode fornecer um único **valor global**, por exemplo $L = 10$.

Porém, esse valor não indica **como ajustar cada peso e bias da rede**.

O gradiente resolve isso calculando, para **cada parâmetro**, sua influência sobre a perda. Assim, cada parâmetro pode ter **direção e intensidade de influência diferentes**; o sinal do gradiente indica a direção local de aumento da perda e, para minimizá-la, o Gradient Descent atua na direção oposta.

$$\frac{\partial L}{\partial w_1}, \quad \frac{\partial L}{\partial w_2}, \quad \cdots, \quad \frac{\partial L}{\partial b_1}, \quad \frac{\partial L}{\partial b_2}, \quad \cdots$$

Assim, podemos ter, por exemplo:
peso $w_1 \rightarrow$ gradiente $+0{,}8$
peso $w_2 \rightarrow$ gradiente $-0{,}3$
bias $b_1 \rightarrow$ gradiente $+0{,}05$
peso $w_3 \rightarrow$ gradiente $+0{,}01$
bias $b_2 \rightarrow$ gradiente $-0{,}4$
...

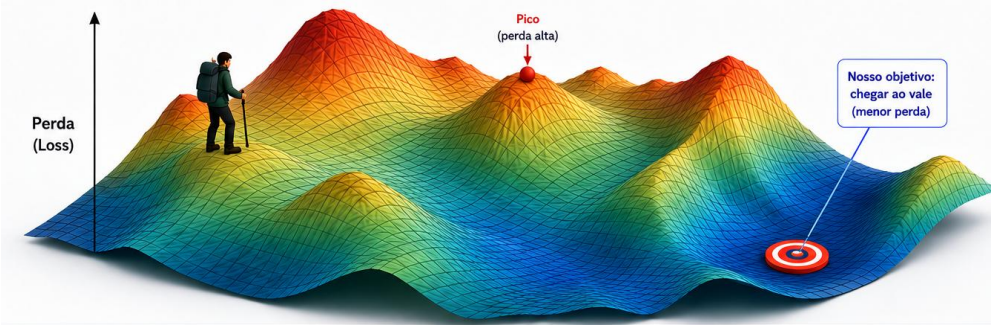


HIPERPARÂMETROS
OTIMIZADOR

A partir do momento em que os gradientes foram calculados, já sabemos como cada peso e *bias* influencia a função de perda. O próximo passo é **utilizar essa informação para atualizar os parâmetros**.

O **otimizador** é o **algoritmo responsável** por determinar **como essa atualização será realizada**, utilizando os gradientes calculados pela retropropagação (*backpropagation*).

O **Gradiente Descendente (*Gradient Descent*)** representa a ideia fundamental desse processo. Diferentes otimizadores utilizam os gradientes de maneiras distintas, buscando melhorar a **velocidade e a estabilidade da convergência**.





HIPERPARÂMETROS
OTIMIZADOR

Entre os principais estão:

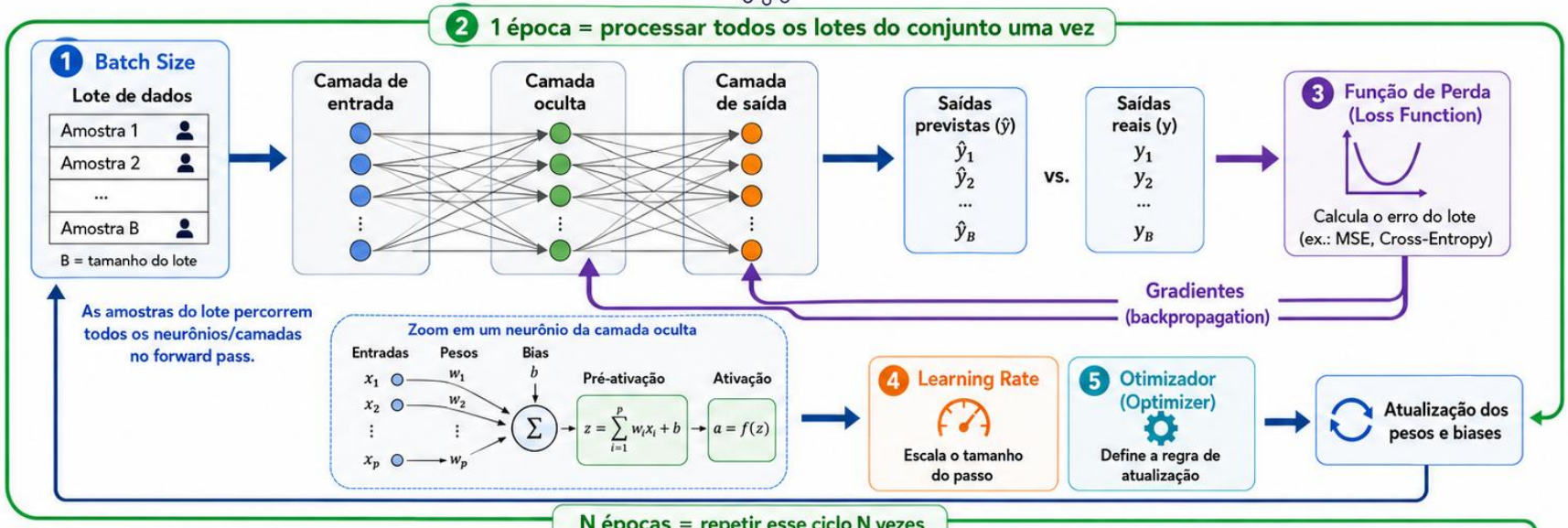
- **SGD (*Stochastic Gradient Descent*)** — no **SGD puro**, a regra é: **"pegue o gradiente, multiplique pela taxa de aprendizado e altere o parâmetro na direção oposta."**

$$w_{novo} = w_{atual} - \eta \cdot g$$
$$w_{novo} = w_{atual} - (0{,}01 \times 0{,}8)$$
$$w_{novo} = w_{atual} - 0{,}008$$

- **SGD com Momentum** — acrescenta ao SGD uma espécie de **"inércia"**, considerando também atualizações anteriores para estabilizar e reduzi oscilações.
- **RMSprop — adapta o tamanho das atualizações para cada parâmetro** com base no histórico recente dos gradientes, reduzindo passos em parâmetros que apresentam gradientes persistentemente maiores.
- **Adam** — combina ideias de **Momentum e RMSprop**, considerando tanto a direção acumulada dos gradientes quanto suas magnitudes recentes. É um dos otimizadores mais utilizados principalmente em redes profundas.
- **AdamW** — variante do Adam que trata de forma separada a **regularização dos pesos (*weight decay*)**, sendo bastante utilizado em modelos modernos.

HIPERPARÂMETROS DE TREINAMENTO EM REDES NEURAIS

Onde cada hiperparâmetro atua durante o treinamento



1 Tamanho do Lote (Batch Size)

- Quantas amostras formam cada lote.
- Atua na entrada de cada ciclo de treinamento.

2 Número de Épocas (Epochs)

- Quantas vezes todos os lotes são processados.
- Atua no ciclo global de treinamento.

3 Função de Perda (Loss Function)

- Mede o erro entre ŷ e y.
- Atua após a camada de saida.

4 Taxa de Aprendizado (Learning Rate)

- Controla a magnitude da atualização.
- Atua na etapa de atualização dos parâmetros.

5 Otimizador (Optimizer)

- Transforma gradientes em atualização dos parâmetros.
- Atua junto com o learning rate.

Fluxo do treinamento: lote → forward pass (rede) → saída (ŷ) → loss → gradientes (backprop) → otimizador + learning rate → atualização (parâmetros)